

ASSESSMENT TOOL 1-APPLICATION BASED QUESTIONS

Course Code: DSA0504

**Course Title: Query Processing
for Data Science**

CO Assessed: CO1

**Assessment: Tool 2 –
Application Based Questions**

Weightage: 25 Marks

Total Marks: 25 Marks

CO2 Statement: Explain the fundamentals of data wrangling, its importance, processes, and tools. Understand different data formats such as CSV, JSON, and XML along with machine-readable data.

Student Name: J.Subhaashree

Reg. No.: 192424008

Section / Batch: 2024

Date of Submission: 31/7/26

**Faculty In-charge: Dr. R.Mahesh
Muthulakshmi**

**Project Mode: Individual
Submission**

1) Movie Database Management System

Aim

To develop a web application that combines movie information from XML files and viewer ratings from CSV files, verifies the data, calculates the average rating for each movie, and generates a sorted report.

Objectives

- To read movie details from an XML file.
- To read viewer ratings from a CSV file.
- To combine the datasets using Movie ID.
- To calculate the average rating for each movie.
- To generate a report sorted by ratings.

Description of the Project

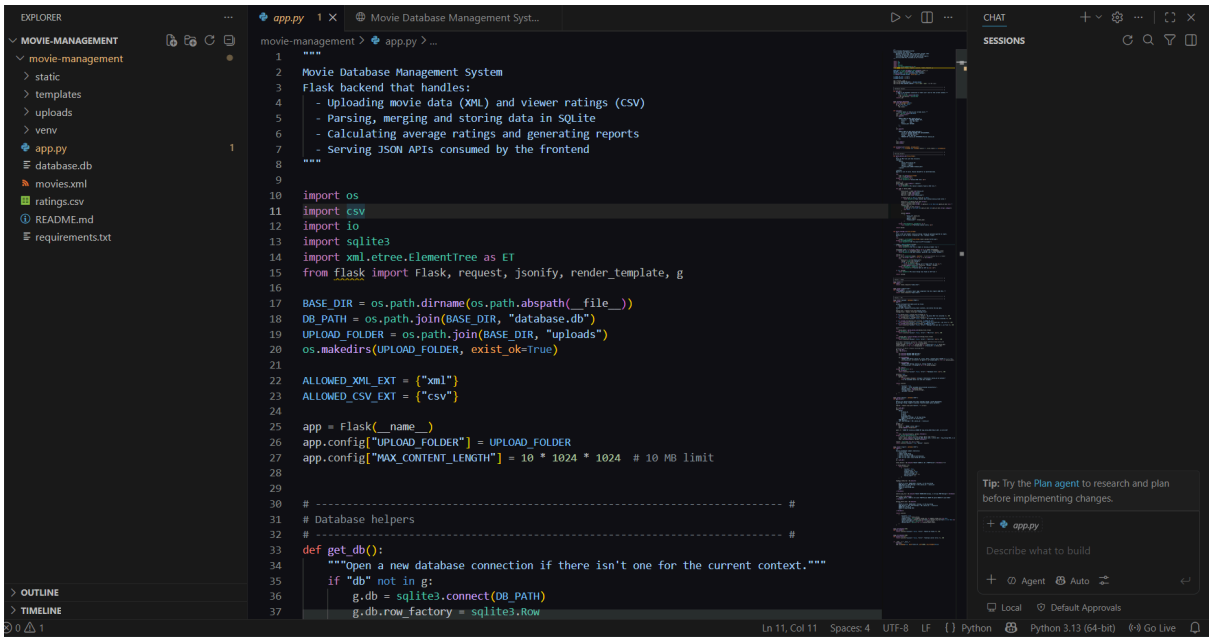
The Movie Database Management System is a web application that manages movie information and viewer ratings. The system imports movie details from an XML file and ratings from a CSV file. It combines the datasets using Movie ID, calculates the average rating for each movie, and displays the results in a sorted report. This application helps streaming platforms analyze movie popularity and viewer preferences.

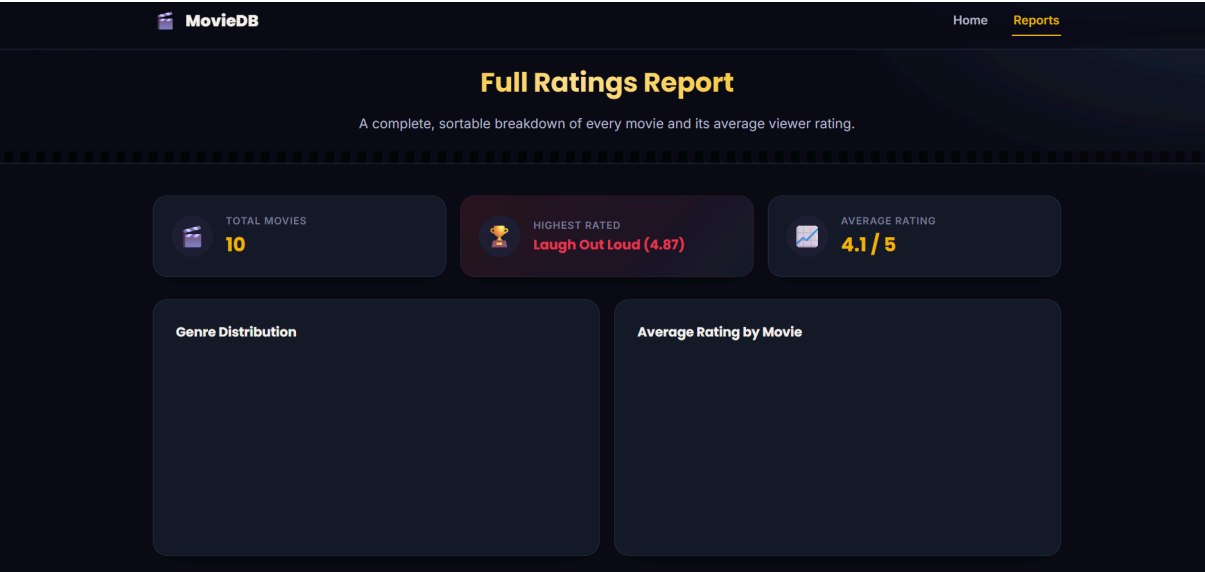
Input

- XML file containing:
 - Movie ID
 - Movie name
 - Genre
 - Release year
- CSV file containing:
 - Movie ID
 - Viewer rating

Output

Program





The table displays a list of movies with their ratings. It includes a search bar at the top right. The data is as follows:

MOVIE ID	MOVIE NAME	GENRE	YEAR	AVERAGE RATING
#3	1 Laugh Out Loud	Comedy	2021	★ 4.87
#10	2 The Last Kingdom	Fantasy	2019	★ 4.80
#5	3 Galaxy Wanderers	Sci-Fi	2022	★ 4.77
#1	The Great Escape	Action	2019	★ 4.43
#9	Speed Legends	Action	2021	★ 4.30
#8	City of Dreams	Drama	2020	★ 4.20
#6	Love in Autumn	Romance	2017	★ 3.90
#2	Silent Waters	Drama	2020	★ 3.77
#4	Midnight Mystery	Thriller	2018	★ 3.23

Applications

- OTT platforms
- Movie recommendation systems
- Entertainment analytics
- Viewer preference analysis
- Rating management systems

Conclusion-The project successfully integrates movie and rating datasets, computes average ratings, and generates a structured report. It simplifies movie analysis and improves decision-making for streaming platforms.

2. Railway Ticket Reservation System

Aim

To develop a web application that combines passenger information from CSV files and train schedules from JSON files, verifies train numbers, and generates a final reservation report.

Objectives

- To read passenger details from a CSV file.
- To read train schedules from a JSON file.
- To verify train numbers for each passenger.
- To combine passenger and train information.
- To generate a reservation report.

Description of the Project

The Railway Ticket Reservation System is a web application that manages passenger reservations and train schedules. Passenger details are stored in a CSV file, while train information is stored in a JSON file. The system verifies whether the train number entered by each passenger exists in the schedule, combines the datasets, and generates a final reservation report indicating valid and invalid reservations.

Input

- CSV file containing:
 - Passenger ID
 - Passenger name
 - Train number
 - Source
 - Destination
- JSON file containing:
 - Train number
 - Train name

- Departure time
- Arrival time

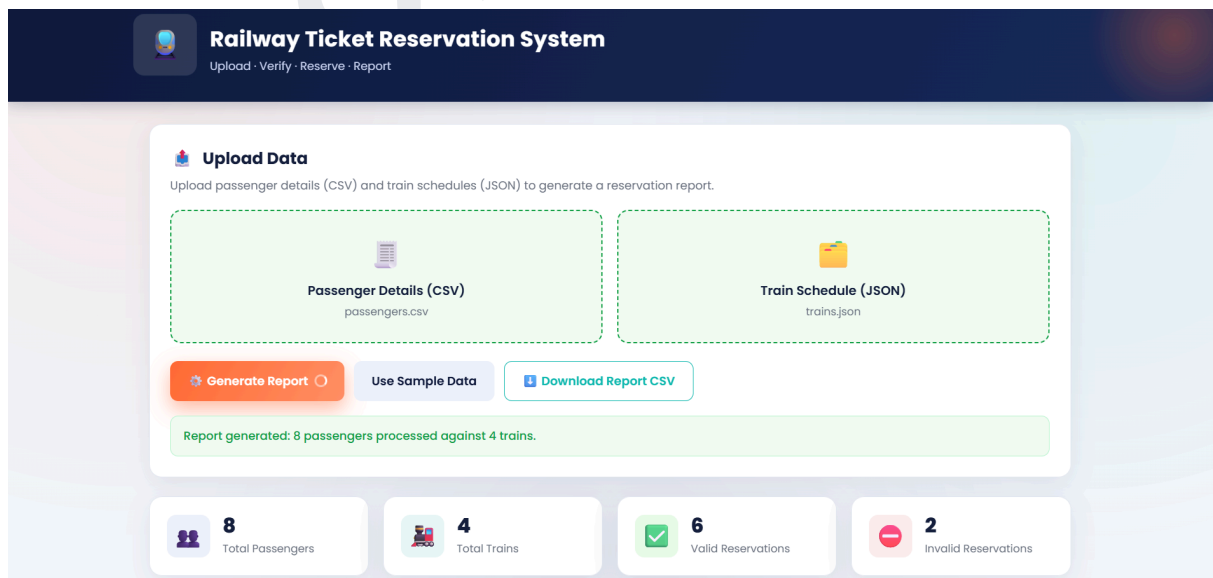
Output

Program

```

1  """
2  Railway Ticket Reservation System
3  -----
4  Full-stack Flask application that:
5  - Accepts passenger details (CSV) and train schedules (JSON) uploads
6  - Stores them in an SQLite database
7  - Cross-verifies each passenger's train number against the train schedule
8  - Produces a merged reservation report (Valid / Invalid)
9  - Exposes REST-style JSON APIs consumed by the frontend
10 """
11
12 import csv
13 import io
14 import json
15 import os
16 import sqlite3
17 from datetime import datetime
18
19 from flask import (
20     Flask,
21     g,
22     jsonify,
23     render_template,
24     request,
25     send_file,
26     send_from_directory,
27 )
28
29 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
30 DB_PATH = os.path.join(BASE_DIR, "database", "railway.db")
31 SAMPLE_DIR = os.path.join(BASE_DIR, "sample_data")
32 os.makedirs(os.path.join(BASE_DIR, "database"), exist_ok=True)
33
34 app = Flask(__name__)
35 app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024 # 5 MB upload limit
36
37

```





Reservation Report

All Statuses

PASSENGER ID	PASSENGER NAME	TRAIN NUMBER	TRAIN NAME	SOURCE	DESTINATION	DEPARTURE	ARRIVAL	STATUS
1	Ravi	101	Chennai Express	Chennai	Bangalore	08:00 AM	02:00 PM	Valid
2	Priya	102	Superfast Express	Madurai	Chennai	10:00 AM	04:00 PM	Valid
3	Arun	999	N/A	Coimbatore	Salem	N/A	N/A	Invalid
4	Divya	103	Trichy Passenger	Chennai	Trichy	06:30 AM	11:00 AM	Valid
5	Karthik	101	Chennai Express	Bangalore	Chennai	08:00 AM	02:00 PM	Valid
6	Meena	104	Erode Intercity	Salem	Erode	01:00 PM	03:30 PM	Valid
7	Suresh	888	N/A	Trichy	Madurai	N/A	N/A	Invalid
8	Lakshmi	102	Superfast Express	Chennai	Madurai	10:00 AM	04:00 PM	Valid

Applications

- Railway reservation systems
- Ticket booking platforms
- Passenger management systems
- Transport scheduling systems
- Reservation verification systems

Conclusion

The project successfully integrates passenger and train datasets, verifies train numbers, and generates a reservation report. It improves reservation accuracy and reduces booking errors.

3)Online Shopping Order Processing System

Aim

To develop a web application that reads customer information from a JSON file and order details from a CSV file, merges the datasets using Customer ID, identifies customers with multiple orders, and generates a consolidated report

Objectives

- To read customer information from a JSON file.
- To read order details from a CSV file.
- To merge customers and order datasets using Customer ID.
- To identify customers who have placed multiple orders.
- To generate a consolidated report in CSV format.

Description of the Application

The Online Shopping Order Processing System is an e-commerce web application that manages customer and order information. Customer details are stored in a JSON file, while order details are stored in a CSV file. The system combines both datasets using Customer ID and identifies customers who have placed multiple orders. Finally, it generates a consolidated report containing complete customer and order details.

Input

JSON file (Customer details):

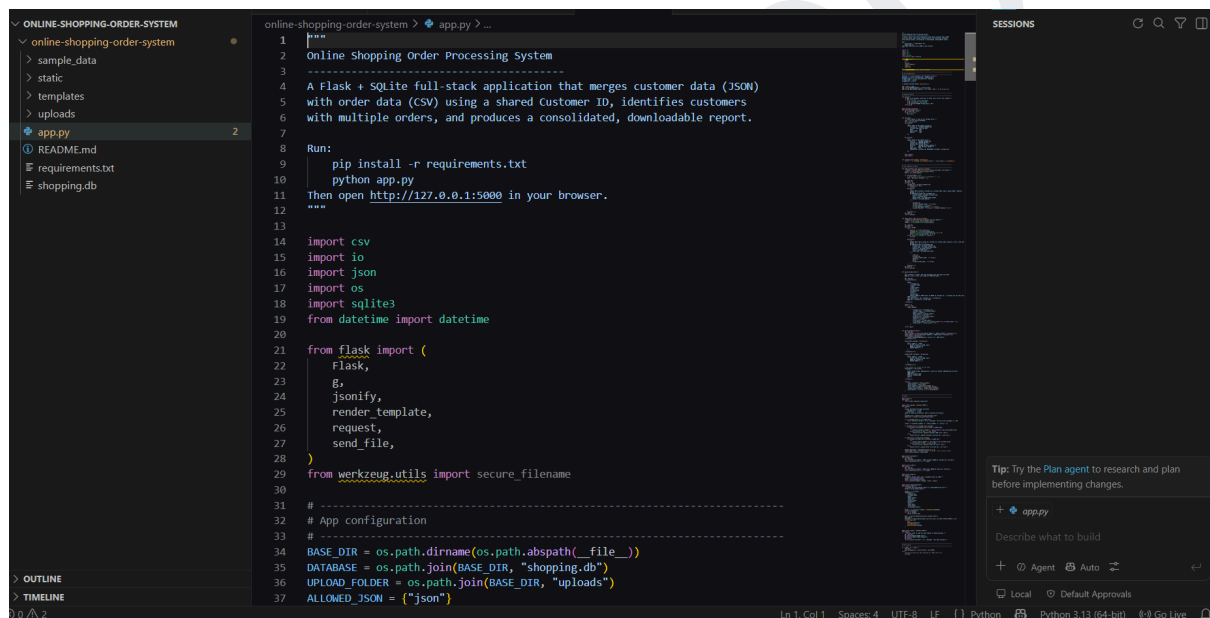
- Customer ID
- Customer Name
- Email ID
- Phone Number
- Address

CSV file (Order details):

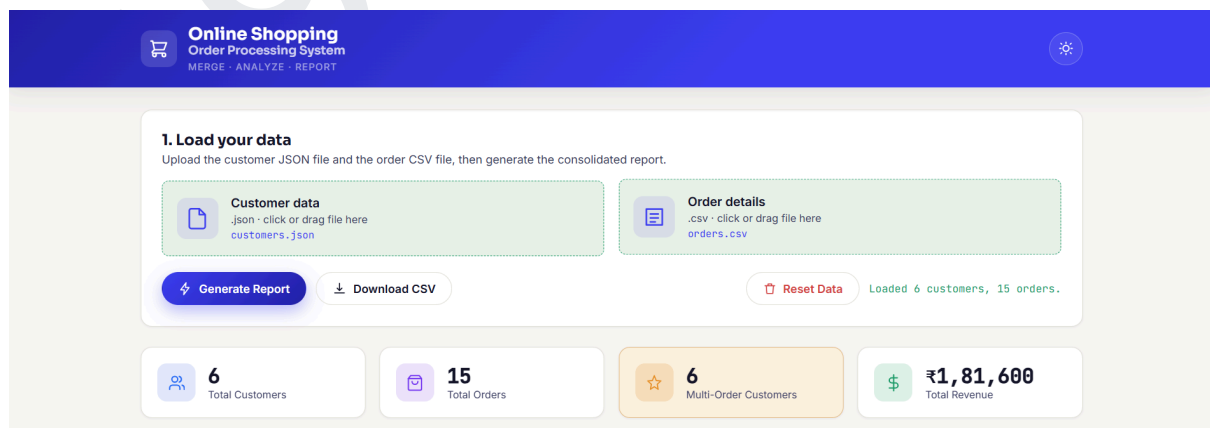
- Order ID
- Customer ID
- Product Name
- Quantity
- Price
- Order Date

Output

Program



```
1 # app.py
2 Online Shopping Order Processing System
3
4 A Flask + SQLite full-stack application that merges customer data (JSON)
5 with order data (CSV) using a shared Customer ID, identifies customers
6 with multiple orders, and produces a consolidated, downloadable report.
7
8 Run:
9   pip install -r requirements.txt
10  python app.py
11  Then open http://127.0.0.1:5000 in your browser.
12
13
14 import csv
15 import io
16 import json
17 import os
18 import sqlite3
19 from datetime import datetime
20
21 from flask import (
22     Flask,
23     g,
24     jsonify,
25     render_template,
26     request,
27     send_file,
28 )
29 from werkzeug.utils import secure_filename
30
31 # -----
32 # App configuration
33 # -----
34 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
35 DATABASE = os.path.join(BASE_DIR, "shopping.db")
36 UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")
37 ALLOWED_JSON = ("json")
```





Online Shopping
Order Processing System
MERGE · ANALYZE · REPORT

Consolidated Order Report

Rows highlighted in amber belong to customers with more than one order.

All Customers

Multiple Orders

Single Order

CUSTOMER ID	CUSTOMER NAME	EMAIL	PHONE NUMBER	ORDER ID	PRODUCT NAME	QUANTITY	PRICE	ORDER DATE
1	Rahul Sharma	rahul@gmail.com	9876543210	101	Laptop	1	₹50,000	2026-07-20
1	Rahul Sharma	rahul@gmail.com	9876543210	102	Wireless Mouse	2	₹1,000	2026-07-21
1	Rahul Sharma	rahul@gmail.com	9876543210	104	USB-C Hub	1	₹2,200	2026-07-25
2	Priya Nair	priya@gmail.com	9876543211	103	Mechanical Keyboard	1	₹1,500	2026-07-21
2	Priya Nair	priya@gmail.com	9876543211	108	27-inch Monitor	1	₹18,000	2026-07-22
2	Priya Nair	priya@gmail.com	9876543211	114	Webcam	1	₹3,200	2026-07-30
3	Arjun Mehta	arjun.mehta@gmail.com	9876543212	106	Phone Case	3	₹300	2026-07-15

4	Sneha Reddy	sneha.reddy@gmail.com	9876543213	107	Bluetooth Headphones	1	₹4,500	2026-07-18
4	Sneha Reddy	sneha.reddy@gmail.com	9876543213	113	Tablet	1	₹22,000	2026-07-29
5	Vikram Singh	vikram.singh@gmail.com	9876543214	109	Office Chair	1	₹8,500	2026-07-10
5	Vikram Singh	vikram.singh@gmail.com	9876543214	110	Standing Desk	1	₹15,000	2026-07-11
5	Vikram Singh	vikram.singh@gmail.com	9876543214	111	Desk Lamp	2	₹900	2026-07-19
6	Ananya Iyer	ananya.iyer@gmail.com	9876543215	112	Smartwatch	1	₹12,000	2026-07-27
6	Ananya Iyer	ananya.iyer@gmail.com	9876543215	115	Fitness Band	2	₹2,500	2026-07-30

Applications

- E-commerce websites
- Online shopping platforms
- Order management systems
- Customer relationship management systems
- Sales and business analytics

Conclusion

The Online Shopping Order Processing System successfully integrates customer and order data from different file formats. It identifies customers with multiple orders and generates consolidated report, making order management and customer analysis easier and more efficient.

4) Employee Payroll Preparation

Aim

To develop a Python application that reads employee information from a CSV file and salary details from a JSON file, merges the datasets using Employee ID, identifies employees with missing salary records, and generates a payroll report.

Objectives

- To read employee data from a CSV file.
- To read salary details from a JSON file.
- To merge the datasets using Employee ID.
- To identify employees with missing salary records.
- To generate a payroll report containing employee and salary details.

Description

The Employee Payroll Preparation System is a Python application used to manage employee and salary information. Employee details are stored in a CSV file, while salary details are maintained in a JSON file. The system reads both datasets and combines them using Employee ID. It checks for employees whose salary information is missing and generates a payroll report containing complete employee details and salary information. This application helps organizations maintain payroll records efficiently and accurately.

Input

CSV File (Employee Details)

- Employee ID
- Employee Name
- Department

JSON File (Salary Details)

- Employee ID
- Salary

Output

Program

```
emp.py - C:/Users/subha/Downloads/emp.py (3.12.10)
File Edit Format Run Options Window Help

employees = []

with open("employee.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        employees.append(row)

# Read salary data from JSON
with open("salary.json", "r") as file:
    salary_data = json.load(file)

# Convert salary list to dictionary
salary_dict = {}

for item in salary_data:
    salary_dict[str(item["employee_id"])] = item["salary"]

print("PAYROLL REPORT")
print("-" * 50)

for employee in employees:
    emp_id = employee["employee_id"]

    if emp_id in salary_dict:
        salary = salary_dict[emp_id]

        print(
            emp_id,
            employee["name"],
            employee["department"],
            salary
        )

    else:
        print(
            emp_id,
            employee["name"],
            employee["department"],
            "Salary Record Missing"
        )
```

```
===== RESTART: C:/Users/subha/Downloads/emp.py =====  
PAYROLL REPORT  
-----  
1 Ravi HR 30000  
2 Priya IT 45000  
3 Arun Finance Salary Record Missing  
4 Meena Marketing 40000  
|
```

Conclusion

The Employee Payroll Preparation System successfully combines employee information from a CSV file and salary details from a JSON file using Employee ID. The system identifies employees with missing salary records and generates an accurate payroll report. This application simplifies payroll management, reduces manual errors, and helps organizations maintain employee salary records efficiently.

5) Mobile Phone Sales Analysis

Aim

To develop a Python application that reads product details from a JSON file and sales transactions from a CSV file, combines the datasets using Product ID, calculates the total quantity sold for each mobile phone, and generates a sales report.

Objectives

- To read product details from a JSON file.
- To read sales transactions from a CSV file.
- To merge the datasets using Product ID.
- To calculate the total quantity sold for each mobile phone.
- To generate a consolidated sales report.

Description

The Mobile Phone Sales Analysis System is a Python application used to analyze mobile phone sales data. Product information such as product ID, mobile name, and brand is stored in a JSON file, while sales transactions are stored in a CSV file. The application combines both datasets using Product ID and calculates the total quantity sold for each product. Finally, it generates a report that helps retailers analyze sales performance and identify popular products.

Input

JSON file (Product details)

- Product ID
- Mobile name
- Brand
- Price

CSV file (Sales transactions)

- Sale ID
- Product ID
- Quantity sold
- Sale date

Output

```
mobile.py - C:/Users/subha/Downloads/mobile.py (3.12.10)
File Edit Format Run Options Window Help
import csv
import json

# Read product data from JSON
with open("products.json", "r") as file:
    products = json.load(file)

# Read sales data from CSV
sales = []

with open("sales.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        sales.append(row)

# Convert product list into dictionary
product_dict = {}

for product in products:
    product_dict[str(product["product_id"])] = {
        "name": product["mobile_name"],
        "brand": product["brand"]
    }

# Calculate total sales
total_sales = {}

for sale in sales:
    product_id = sale["product_id"]
    quantity = int(sale["quantity"])

    if product_id in total_sales:
        total_sales[product_id] += quantity
    else:
        total_sales[product_id] = quantity

# Print report
```



```
===== RESTART: C:/Users/subha/Downloads/mobile.py =====
```

MOBILE PHONE SALES REPORT

```
-----  
Product ID: 101 | Mobile: Samsung Galaxy S24 | Brand: Samsung | Total Sold: 7  
Product ID: 102 | Mobile: iPhone 15 | Brand: Apple | Total Sold: 4  
Product ID: 103 | Mobile: OnePlus 12 | Brand: OnePlus | Total Sold: 4  
|
```

Conclusion

The Mobile Phone Sales Analysis System successfully combines product and sales data from different file formats and generates a detailed sales report. It helps retailers analyze sales trends, identify best-selling products, and improve inventory management and business decisions.